

K. Krishna

Priya

What is NumPy?

- 💡 NumPy is a Python package.
- 💡 It stands for '**Numerical Python**'. It is a library consisting of multidimensional array objects and

a collection of routines for processing of array.

💡 NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

💡 It also has functions for working in domain of linear algebra, fourier transform, and matrices.

Why Use NumPy?

💡 In Python we have lists that serve the purpose of arrays, but they are slow to process.

💡 NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.

💡 The array object in NumPy is called ndarray, it provides a lot of supporting functions that make

working with ndarray very easy.

- 💡 Arrays are very frequently used in data science, where speed and resources are very important.

Operations using NumPy

- 💡 Using NumPy, a developer can perform the following operations –

- 💡 Mathematical and logical operations on arrays.

- 💡 Fourier transforms and routines for shape manipulation. 💡 Operations related to linear algebra. NumPy has in-built functions for linear algebra and random number generation.

`numpy.array`

💡 It creates a ndarray from any object exposing an array interface, or from any method that returns an array.

💡 **`numpy.array(object, dtype = None, copy = True, order = None, subok = False, ndmin = 0)`**

Why is NumPy Faster Than Lists?

💡 NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.

💡 This behavior is called locality of reference in computer science.

💡 This is the main reason why NumPy is faster

than lists. Also it is optimized to work with latest CPU architectures. 🧠 To create an ndarray, we can pass a list, tuple or any array like object into the `array()` method, and it will be converted into an ndarray:

Features of NumPy

- 🧠 NumPy has various features which make them popular over lists.
- 🧠 Provides powerful, multidimensional array objects (ndarrays).
- 🧠 Supports advanced slicing and indexing techniques. 🧠 Reduces the need for explicit loops with efficient array operations.

- 💡 Easily integrates with other libraries like pandas, matplotlib, and SciPy.
- 💡 Useful linear algebra, Fourier transform, and random number capabilities

Installation of NumPy

- 💡 Install it using this command:
- 💡 `C:\Users\Your Name>pip install numpy`
- 💡 **Create a NumPy ndarray Object**
- 💡 NumPy is used to work with arrays. The array object in NumPy is called ndarray.
- 💡 We can create a NumPy ndarray object by using the `array()` function. 💡 **Example**
- 💡 `import numpy as np`

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

```
print(type(arr))
```

💡 **type():** This built-in Python function tells us the type of the object passed to it. Like in above code it shows that arr is numpy.ndarray type.

Use a tuple to create a NumPy array:

💡 `import numpy as np`

```
arr = np.array((1, 2, 3, 4, 5))
```

```
print(arr)
```

0-D Arrays

💡 0-D arrays, or Scalars, are the elements in

an array. Each value in an array is a 0-D array.

💡 **Example**

💡 Create a 0-D array with value 42

💡 `import numpy as np`

```
arr = np.array(42)
```

```
print(arr)
```

1-D Arrays

💡 An array that has 0-D arrays as its elements is called uni dimensional or 1-D array.

💡 These are the most common and basic arrays. 💡 **Example**

💡 Create a 1-D array containing the values 1,2,3,4,5: 💡 import numpy as np

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

Create a 2-D array containing two arrays

💡 **Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:**

💡 import numpy as np

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(arr)
```

3-D arrays

💡 An array that has 2-D arrays (matrices) as its elements is called 3-D array.

💡 **Example**

💡 Create a 3-D array with two 2-D arrays, both containing two arrays with the values 1,2,3 and 4,5,6:

💡 `import numpy as np`

```
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3],
```

```
[4, 5, 6]]) print(arr)
```

Higher Dimensional Arrays

💡 An array can have any number of dimensions. 💡 When the array is created, you can define the number of dimensions by using the `ndmin` argument.

💡 Example

💡 Create an array with 5 dimensions and verify that it has 5 dimensions:

```
💡 import numpy as np                                :, arr.ndim)  💡 print('shape  
of array :', arr.shape)
```

```
arr = np.array([1, 2, 3, 4],  
ndmin=5)
```

```
print(arr)
```

```
print('number of dimensions
```

Check Number of Dimensions?

💡 NumPy Arrays provides the `ndim` attribute that returns an integer that tells us how many dimensions the array have.

💡 Example

💡 Check how many dimensions the arrays have:

💡 `import numpy as np`

```
a = np.array(42)
```

```
b = np.array([1, 2, 3, 4, 5]) print(c.ndim)
```

```
c = np.array([[1, 2, 3], [4, 5, 6]]) print(d.ndim)
```

```
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
```

```
print(a.ndim)
```

```
print(b.ndim)
```

💡 In this array the innermost dimension (5th dim) has 4 elements, the 4th dim has 1 element that is the vector, the 3rd

dim has 1 element that is the matrix with the vector, the 2nd dim has 1 element that is 3D array and 1st dim has 1 element that is a 4D array.

NumPy – Data Types

bool_ Boolean (True or False) stored as a byte

int_ Default integer type (same as C long; normally either int64 or int32)

intc Default integer type (same as C long; normally either int64 or int32)

intp An integer used for indexing (same as C ssize_t; normally either int32 or int64)

int8 Byte (-128 to 127)

int16 Integer (-32768 to 32767)

float_ Shorthand for float64

float64 Double precision float: sign bit, 11 bits exponent, 52 bits

mantissa **float64** Double precision float: sign bit, 11 bits exponent, 52

bits mantissa **complex_** Shorthand for complex128

complex64 Complex number, represented by two 32-bit floats (real and imaginary components)

complex128 Complex number, represented by two 64-bit floats (real and imaginary components)

Data Type Objects (dtype)

- 💡 A data type object describes the interpretation of a fixed block of memory corresponding to an array, depending on the following aspects –
 - 💡 Type of data (integer, float or Python object)
 - 💡 Size of data
 - 💡 Byte order (little-endian or big-endian)
 - 💡 In case of structured type, the names of fields, data type of each field and part of the memory block taken by each field. 💡 If the data type is a subarray, its shape and data type

ndarray.shape

💡 This array attribute returns a tuple consisting of array dimensions. It can also be used to resize the array. 💡 **Example:**

💡 `import numpy as np`

💡 `a = np.array([[1,2,3],[4,5,6]])`

💡 `print a.shape`

💡 The **output** is as follows `-(2, 3)`

resizes the ndarray

💡 **# Example** this resizes the

ndarray 💡 `import numpy as np`

💡 `a = np.array([[1,2,3],[4,5,6]])`

💡 `a.shape = (3,2)`

💡 print a

💡 The output is as follows -`[[1, 2][3, 4] [5, 6]]`

`ndarray.ndim`

💡 This array attribute returns the number of array dimensions. 💡 this is one dimensional array

💡 import numpy as np 💡 # b is having three dimensions

💡 `a = np.arange(24)`

💡 `a.ndim`

💡 # now reshape it

💡 `b =`

`a.reshape(2,4,3)`

💡 print b

Reshaping arrays

- 💡 Reshaping means changing the shape of an array.
- 💡 The shape of an array is the number of elements in each dimension. 💡 By reshaping we can add or remove dimensions or change number of elements in each dimension.
- 💡 We can use **reshape** method to reshape an array.
- 💡 Consider an array with shape (a1, a2, a3, ..., aN). We can reshape and convert it into another array with shape (b1, b2, b3, ..., bM). The only required condition is $a1 \times a2 \times a3 \dots \times aN = b1 \times b2 \times b3 \dots \times bM$. (i.e. the original size of the array remains unchanged.)

💡 Example

- 💡 Convert the following 1-D array with 12 elements into a 2-D array. 💡 The outermost dimension will have 4 arrays, each with 3 elements: 💡 `import numpy as np`

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(4, 3)
```

`print(newarr)`

numpy.zeros

💡 Returns a new array of specified size, filled with zeros. 💡 `numpy.zeros(shape, dtype = float, order = 'C')` 💡 `# array of five ones.`

Default dtype is float 💡 `import numpy as np`

💡 `x = np.ones(5)`

💡 `print x`

💡 The output is as follows –

💡 `[ 1. 1. 1. 1. 1.]`

Providing Negative Arguments

💡 If you provide negative values for start or both

start and stop, and have a positive step, then `arange()` will work the same way as with all positive arguments:

💡 Example:

```
np.arange(-5, -1)
```

```
o/p: array([-5, -4, -3, -2])
```

💡 -----

```
💡 >>> np.arange(-8, -2, 2)
```

```
💡 array([-8, -6, -4])
```

```
💡 >>> np.arange(-5, 6, 4)
```

```
💡 array([-5, -1, 3])
```

NumPy – Indexing & Slicing

```
💡 import numpy as np
```

```
a = np.arange(10)
```

```
b = a[2:7:2]
```

```
print b
```

Here, we will get the same output

– [2 4 6] 🧠 **Example:** # slice items

between indexes import numpy as

np

```
a = np.arange(10)
```

```
print a[2:5]
```

Here, the output would be –

[2 3 4]

Counting Backwards

🧠 Sometimes you'll want an array with the values decrementing from left to right. In such cases, you can use `arange()` with a negative value for

step, and with a start greater than stop:

💡 **Example:**

```
np.arange(5, 1, -1)
```

```
array([5, 4, 3, 2])
```

```
array([7, 4, 1])
```

Getting Empty Arrays

💡 There are several edge cases where you can obtain empty NumPy arrays with `arange()`. These are regular instances of `numpy.ndarray` without any elements.

💡 **Examples:-**

```
💡 >>> np.arange(2, 2)
```

```
array([], dtype=int64)
```

💡 >>> np.arange(8, 2, 1)

array([], dtype=int64)

💡 >>> np.arange(2, 8, -1)

array([], dtype=int64)

Create Using arange() Function

💡 This function returns evenly spaced values within a given interval. **Step** size is specified.

💡 # Create a sequence of integers

💡 # from 0 to 30 with steps of 5

💡 f = np.arange(0, 30, 5)

💡 print ("A sequential array with steps of 5:\n", f) 💡 Output:

💡 A sequential array with steps of 5:

💡 [0 5 10 15 20 25]

More Examples:

<code>np.arange(start=1, stop=10, step=3)</code>	<code>array([1, 4, 7])</code>
<code>np.arange(1, 10, 3)</code>	<code>array([1, 4, 7])</code>
<code>np.arange(1, 10, 3, dtype=float)</code>	<code>array([1., 4., 7.])</code>
<code>np.arange(1.0, 10, 3)</code>	<code>array([1., 4., 7.])</code>
<code>np.arange(0, 1.1, 0.5)</code>	<code>array([0. , 0.5, 1.])</code>
<code>np.arange(2, 6)</code>	<code>array([2, 3, 4, 5])</code>
<code>np.arange(5)</code>	<code>array([0, 1, 2, 3, 4])</code>
<code>np.arange(-8, -2, 2)</code>	<code>array([-8, -6, -4])</code>

<code>np.arange(7, 0, -3)</code>	<code>array([7, 4, 1])</code>
<code>np.arange(8, 2)</code>	<code>array([])</code>

Joining NumPy Arrays

💡 Joining means putting contents of two or more arrays in a single array. 💡 We pass a sequence of arrays that we want to join to the `concatenate()` function, along with the axis. If axis is not explicitly passed, it is taken as 0. 💡 Join two 2-D arrays along rows (`axis=1`):

💡 `import numpy as np`

```
a = np.array([[1, 2], [3, 4]])
```

```
b = np.array([[5, 6], [7, 8]])
```

```
arr = np.concatenate((a, b), axis=1)
```

```
print(arr) (or) np.hstack((a, b)) 💡
```

```
np.vstack((b,  
a))
```

Create Using linspace() Function

💡 **linspace()**: It returns evenly spaced values within a given interval.

💡 # Create a sequence of 10 values in range 0 to 5

💡 `g = np.linspace(0, 5, 10)`

💡 `print ("A sequential array with 10 values between"`

💡 `"0 and 5:\n", g)`

💡 Output:

💡 A sequential array with 10 values between 0 and 5:

💡 `[0. 0.55555556 1.11111111 1.66666667 2.22222222 2.77777778`

💡 3.33333333 3.888888889 4.44444444 5.]

advantage of two important
concepts at once:

💡 Vectorization

💡 Broadcasting

💡 Vectorization is the process of performing the same operation in the same way for each element in an array. This removes for loops from your code but achieves the same result.

💡 Example 1

💡 `import numpy as np`

💡 `a = np.array([1,2,3,4])`

💡 `b = np.array([10,20,30,40])`

💡 `c = a * b`

💡 `print c`

💡 Its output is as follows – [10 40 90 160]

NumPy – Broadcasting

- 💡 the term **broadcasting** refers to the ability of NumPy to treat arrays of different shapes during arithmetic operations. Arithmetic operations on arrays are usually done on corresponding elements.
- 💡 The smaller array is **broadcast** to the size of the larger array so that they have compatible shapes.
- 💡 Broadcasting is the process of extending two arrays of different shapes and figuring out how to perform a vectorized calculation between them. Remember, `grades` is an array of numbers of shape `(8,)` and `change` is a scalar, or single number, essentially with shape `(1,)`. In this case, NumPy adds the scalar to each item in the array and returns a new array with the results.

Example for broadcasting

```
import numpy as np
a = np.array([[0.0,0.0,0.0],[10.0,10.0,10.0],[20.0,20.0,20.0],
              [30.0,30.0,30.0]])
b = np.array([1.0,2.0,3.0])
print 'First array:'
print a
print 'Second array:'
print b
print 'First Array + Second Array'
print a + b
```

Sorting Arrays

🧠 Sorting means putting elements in an

ordered sequence. 🧠 *Ordered sequence* is any sequence that has an order corresponding to elements, like numeric or alphabetical, ascending or descending.

🧠 The NumPy ndarray object has a function called `sort()`, that will sort a specified array.

🧠 `import numpy as np`

🧠 `arr = np.array([[3,`

`2, 4], [5, 0, 1]])` 🧠

`print(np.sort(arr))`

Transposing and

Concatenating

🧠 Here's **transposing**
an array

💡 `a = np.array([ [1, 2], [3, 4], [5, 6] ])`

💡 `a.T`

💡 Out: `array([[1, 3, 5],
[2, 4, 6]])`

💡 `a.transpose()`

💡 Out: `array([[1, 3, 5],
[2, 4, 6]])`

Splitting NumPy Arrays

💡 Splitting is reverse operation of Joining.

💡 Joining merges multiple arrays into one and Splitting breaks one array into multiple.

💡 We use `array_split()` for splitting arrays, we pass it the array we want to split and the number of splits.

💡 If the array has less elements than required, it will

adjust from the end accordingly.

```
💡 import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6])
```

```
newarr = np.array_split(arr, 4)
```

```
print(newarr)
```

Creating the Filter Array

💡 In the example above we hard-coded the True and False values, but the common use is to create a filter array based on conditions. 💡 **Example**

💡 Create a filter array that will return only values higher than 42: 💡 import numpy as np

```
arr = np.array([41, 42, 43, 44])
```

```
# Create an empty list
```

```

filter_arr = []
# go through each element in arr
for element in arr:
    # if the element is higher than 42, set the value to True,
    # otherwise False: if element > 42:
        filter_arr.append(True)
        print(filter_arr)
    else:
        filter_arr.append(False)
        print(newarr)
else) newarr =
arr[filter_arr]

```

Masking and Filtering

💡 Index-based selection is great, but what if you want to filter your data based on more complicated nonuniform or nonsequential criteria? This is where the concept of a mask comes into play.

💡 A mask is an array that has the exact same shape as your data, but instead of your values, it holds Boolean values: either True or False. 💡 **numbers = np.linspace(5, 50, 24, dtype=int).reshape(4, -1)** 💡 np.linspace() generates n numbers evenly distributed between a minimum and a maximum, which is useful for evenly distributed sampling in scientific plotting.

💡 Finally, array.reshape() can take -1 as one of its dimension sizes. That signifies that NumPy should just figure out how big that particular axis needs to be based on the size of the other axes. In this case, with 24 values and a size of 4 in axis 0, axis 1 ends up with a size of 6.

Generate Random Array

💡 Random number does NOT mean a different number every time. Random means something that can not be predicted logically. 💡 NumPy offers the **random**

module to work with random numbers.

- 💡 The random module's **rand()** method returns a random float between 0 and 1.

- 💡 The **randint()** method takes a size parameter where you can specify the shape of an array.

- 💡 Generate a 2-D array with 3 rows, each row containing 5 random integers from 0 to 100:

- 💡

```
from numpy import random  
x = random.randint(100, size=(3, 5))  
print(x)
```

Generate Random Number From Array

- 💡 The **choice()** method allows you to generate a random value based on an array of values.

- 💡 The **choice()** method takes an array as a parameter and randomly returns one of the

values.

💡 Generate a 2-D array that consists of the values in the array parameter (3, 5, 7, and 9):

💡

```
from numpy import random  
x = random.choice([3, 5, 7, 9], size=(3, 5))  
print(x)
```